VeriCommerce AI

Evidence-grounded commerce intelligence

Comprehensive Technical Overview · Zahidul Islam · 15 October 2026

1. Purpose, product and engineering direction

VeriCommerce AI turns product information into a system of traceable evidence. Its purpose is to help people understand what a catalog actually documents, find the passage behind an answer, review content against current sources, and retain the provenance of an approved output. The project is developed and operated by Zahidul Islam.

Commerce information changes across product variants, manuals, policies and catalog updates. A fluent description can still contain the wrong specification. A citation can point to an old revision. A draft can become outdated after someone reviews it. VeriCommerce AI addresses these problems by making source identity, version and approval state part of the application rather than treating them as optional additions to generated text.

The current system ingests structured JSON catalogs, stores immutable source revisions, searches current passages, returns quotations with citations, creates evidence-based draft jobs, checks approval against current sources and exports approved content as JSON. It also exposes provenance relationships, audit metadata and actual measured regression checks.

The interface combines an editorial public website with a server-backed evidence workspace and a separate browser-storage workspace. Product imagery and starting catalog records are generated test material. They do not represent real customer deployments, orders, endorsements or independently verified product properties.

Access currently uses temporary cookie-bound workspaces with a configured 12-hour lifetime. Account login, account recovery, team roles, Shopify authorization, document/image uploads and model training are not integrated capabilities. Export useful work before expiry and keep personal, regulated or confidential records out of the service.

The long-term engineering direction is a reusable evidence platform with commerce as its first application. Retrieval, model engineering and guarded publishing should earn their place through measured outcomes. The roadmap does not require every AI framework in every request. It requires reproducible datasets, trustworthy boundaries, comparisons against baselines and release evidence that a reviewer can assess.

Website: https://vericommerce.zahidul-islam.com/

Evidence workspace: https://vericommerce.zahidul-islam.com/engine

Repository: https://github.com/Zahidulislam2222/VeriCommerce-AI

2. User journeys and the evidence lifecycle

A user begins with a generated catalog or permitted non-sensitive records. Each product has an identifier and descriptive fields; each claim identifies a source, passage and page. A source record has metadata and a version. The application validates references before a durable ingestion job applies the supplied changes.

Imports are incremental. Supplying a record updates it; omitting it does not delete it. An identical replay preserves the existing revision. Changed source content creates an immutable new revision and keeps the relationship to the preceding version. This distinction makes retries safe while retaining evidence of what changed.

A source passage stays connected to its product and revision. Search operates on current evidence in the user's workspace. Answers quote retrieved text and identify the source version and page. A source-inspection dialog lets the user inspect the supporting passage without losing the surrounding review context.

Draft creation is asynchronous. The API records a job, the worker claims it, and the user can inspect the resulting persisted draft. Drafts use an evidence-only template. They are not unrestricted language-model completions, and the current API does not accept arbitrary edited draft text.

Approval is an explicit decision about the draft and its evidence. Before recording approval, the application checks that the supporting revisions are still current. A source or product-metadata change invalidates dependent drafts. An already open approval dialog can therefore encounter a conflict; the interface must explain that conflict and require a fresh review rather than quietly treating the old draft as valid.

An approved export contains compatibility metadata and the draft, including product ID/name and citations. It does not contain a full product record, and it does not publish to an external storefront. This boundary matters: an exported artifact is available for the user's next step, while Shopify writes require a future separately authorized workflow.

The same evidence structure supports change-impact analysis. Relationships show which product is documented by a source and which draft derives from evidence. A new manual revision can identify affected content without rewriting history. The current graph is explicit provenance data, not a trained graph model or a claim that community-summary GraphRAG has been evaluated.

The commercial value of this lifecycle is accountable content handling. Source-backed information can still be wrong if the source is wrong, but the system can show its origin, detect supersession and prevent an old approval from silently authorizing new facts.

3. Architecture, isolation and persistence

The implemented application uses React, TypeScript and Vite for the interface; FastAPI and Pydantic for the API; PostgreSQL for durable evidence and work state; and a separate worker for ingestion and drafting. A pinned ONNX encoder supplies optional CPU embedding ranking. The existing server hosts an isolated container stack behind a same-origin HTTPS gateway.

The browser requests the API through the same public origin. The web container serves static assets and forwards the API prefix. PostgreSQL is not published as a public database endpoint. API and worker use restricted runtime credentials; migrations use a separate administrator credential path. Secrets are mounted outside images and release archives.

Each browser receives an opaque HttpOnly, Secure, SameSite cookie selecting its temporary workspace. The server stores the token hash. This is a scoped access mechanism, not a durable merchant identity system. Clearing the cookie can lose access without immediately deleting stored server records. The worker removes expired workspaces and dependent live records.

Every tenant transaction sets its local context. Forced PostgreSQL row-level security adds a database boundary alongside application filters. The runtime role is checked to ensure it is neither a superuser nor a role that bypasses RLS. Entity lookups remain tenant-scoped, so a valid identifier from another workspace does not become an authorization shortcut.

Input limits, allowed origins, rate bounds and workspace quotas constrain admission and storage. These quotas are configuration values, not evidence of throughput or simultaneous-user capacity. Raising them requires load and abuse review.

Jobs persist with state, attempt, lease and retry information. A worker must verify that it still owns the claimed attempt under a row lock before producing, completing or failing work. This fences a delayed worker that resumes after another attempt has already taken ownership. Database fencing does not imply exactly-once side effects across an external API; future publishing needs reconciliation and its own idempotency design.

Containers run with nonroot users, read-only roots, dropped capabilities, health checks and resource limits. Owned volumes and temporary filesystems provide only the writable paths needed. Dynamic IP allocation excludes the reserved trusted proxy address, and all enabled Nginx temporary directories must use writable paths.

The current server is one failure domain. Isolation and immutable releases improve operational control, but they do not establish high availability, multi-region failover or protection from a fully compromised host administrator.

Reference: PostgreSQL row security — https://www.postgresql.org/docs/current/ddl-rowsecurity.html

4. Retrieval, model behavior and context

The current retrieval system combines PostgreSQL keyword matching with optional local embedding ranking. Keyword search identifies relevant subject matter; the quantized ONNX model adds semantic ordering through reciprocal rank fusion. Dense similarity alone cannot authorize a factual answer.

This conservative boundary has a trade-off. It reduces the chance of answering from an unrelated semantic neighbor, but purely semantic questions can miss useful evidence when they have no lexical subject match. The system reports its actual retrieval method instead of suggesting that every request used an advanced model.

The current encoder is the BGE-small-en-v1.5 ONNX conversion published by Qdrant. Its repository, full revision, model file, cache and thread/batch settings are configuration. The reviewed artifact is pinned to an immutable revision. Normal API and worker startup use the cache; downloading is an explicit preparation operation. Missing optional cache produces an honest keyword-only status.

A read-only runtime requires every downloader cache to be placed under an owned writable location. A warm cache can conceal a broken first-download path, so empty-cache recovery must be tested separately. Model availability is a readiness fact to observe, not an assumption from a local notebook.

Answers are source quotations with revision and page citations. They abstain when relevant evidence is absent. They do not perform complete question-level entailment checking: a quote may be relevant while failing to answer every part of a complex question. Provenance and truthfulness are related but different responsibilities.

The current release does not train models. It does not claim supervised fine-tuning, DPO, GRPO, distillation or improved recommendation quality. Those experiments remain in the separate model-engineering roadmap.

The future retrieval plan compares keyword, dense, hybrid and reranked methods on the same frozen questions. Context construction will select evidence by relevance, authority, freshness, permissions and a bounded budget. Durable preferences, task state and generated summaries must remain separate from verified product evidence.

Training will target extraction, output structure, tool behavior and appropriate abstention rather than memorizing changing price or inventory. It requires permitted data, product-family/source splits, synthetic-label disclosure, duplicate/leakage checks, base-versus-adapted results and documented regressions. A model that fails to improve can remain unpromoted; an honest negative result is useful engineering evidence.

Model sources: https://huggingface.co/BAAI/bge-small-en-v1.5 and https://huggingface.co/Qdrant/bge-small-en-v1.5-onnx-Q

5. Interface, multimodal work and accessibility

The visual system uses warm ivory, oxblood, olive evidence accents, editorial typography and sculptural product imagery. Cormorant Garamond and DM Sans are self-hosted. The interface aims to make evidence relationships readable while keeping navigation, review and recovery actions concrete.

The public website includes platform, pricing, about, contact, documentation, changelog, security, accessibility, legal hub, privacy, terms, cookies, United States and European Union information. Header and grouped footer links have real destinations. Mobile navigation is a disclosure using native links, expanded state, Escape dismissal and focus restoration.

Dialogs retain keyboard focus and return it to their trigger. Inspecting a source from a review returns to that review. Errors remain visible inside the relevant dialog, including changed-source approval conflicts. Long schema-valid imported text is tested at narrow widths instead of relying only on carefully formatted starting records.

Motion preferences are subscribed while the page is open. Switching to reduced motion removes the film and disables smooth scrolling. The silent hero film can be paused and also stops when offscreen or when the document is hidden. Interaction does not depend on animation.

The three product studies were generated with the built-in image tool. Rendered images are WebP derivatives, with PNG sources available for revision. The five-second film is a local FFmpeg animation of the headphone image; it is not a Seedance-generated film. An image of a product cannot establish a hidden technical specification or certification.

WCAG 2.2 AA is an engineering target, not an independent conformance certificate. Automated axe and keyboard checks are useful evidence, but manual screen-reader, zoom, reflow, high-contrast, touch and cognitive reviews remain necessary. Generated descriptions require human judgment.

Future multimodal work separates visible observations, verified attributes and the published description appropriate to a particular purpose. Informative image alternatives convey essential meaning; decorative images use empty alternatives; functional images describe the action. Extensive technical information belongs in visible content, not every alt attribute.

The current service does not ingest PDFs/images or perform OCR/vision understanding. Those features require bounded parsing, resource/time limits, rights checks and truthful error states. They must not appear as working controls before their API and review behavior exist.

References: https://www.w3.org/TR/WCAG22/ and https://www.w3.org/WAI/tutorials/images/

6. Commerce integration and guarded publishing

The current evidence workflow is useful before a storefront connector exists. It provides an explicit boundary for the next commercial steps: who may connect a store, which data may be read, how updates are ordered, what a human approves, and what happens when a remote write has an uncertain outcome.

The planned Shopify connection begins read-only. OAuth state/callback validation, least-privilege scopes, encrypted token storage, disconnect/uninstall handling and verified HMAC webhook receipts precede synchronization. Duplicate and out-of-order events require persisted receipt/order logic, not an assumption that webhooks arrive once.

A sync must preserve variant distinctions, authoritative units and source versions. Source deletion should propagate through passages, indexes, graph edges, caches and dependent drafts. Omitted import rows are not deletion in the current JSON API; the future deletion contract must be explicit.

Product content work includes FAQs, descriptions, purpose-aware alt review and deterministic structured data. Price, availability and shipping must come from current authoritative inputs. A language model should not invent JSON-LD values or review aggregates. Any structured product data must remain consistent with visible content.

The future publishing workflow binds approval to tenant, actor, exact draft hash, source versions, target fields and expiry. A model proposes; application code validates authority; a person approves consequential changes. Publishing credentials stay outside the model's tool boundary.

A crash after a remote mutation is a difficult case. The system must reconcile the current field value before retrying. An uncertain timeout is not proof that no write occurred. Where a remote mutation lacks atomic compare-and-swap, document the race and use detect-and-reconcile or manual export for unsafe operations.

Reversal must respect field ownership. Store before/after values and reverse only if the current value still matches the operation's output. A merchant's later edit should trigger reconciliation rather than being overwritten by an automated rollback.

Current access has no merchant accounts, membership roles or connector credentials. Account recovery, owner/editor/reviewer/viewer permissions and real permitted development-store acceptance remain phase gates. The presence of a roadmap or archived implementation work is not evidence those capabilities are live.

AI-search visibility is a quality/crawlability question, not a promise that an assistant will recommend a product. Measure accepted content and search usefulness with qualified experiments; do not claim placement or conversion uplift from generated events.

7. Evaluation, release evidence and engineering quality

Verification is organized around behavior, not framework count. Criteria come first, implementation stays scoped, required gates pass, and a fresh reviewer checks the result. The application is exercised through real HTTP/UI paths against PostgreSQL and the CPU model rather than relying solely on mocked responses.

The dated evidence-core release passed 24 local PostgreSQL/HTTP/ONNX integration checks, 28 public browser checks, 23 deployment-safety regressions and five domain/configuration checks. Types, lint, security scans, builds and independent review passed. Those counts describe that release, not every future source revision.

The later public-page release passed eight acceptance criteria, eight domain checks, 32 local browser cases and 44 actual public desktop/mobile cases with no skips, flakes or failures. Fifteen deployment-safety regressions and a scoped visual refresh also passed. Its 84 prepared/archive/live artifact hashes matched, while the existing backend image and service identities remained unchanged.

Browser behavior included citations, unsupported answers, durable approval/export, changed-source invalidation, concurrent approval conflict, expired-cookie recovery, maximum-length imports, dialog focus, navigation, reduced motion and accessibility. A direct unknown route returned a branded HTTP404 with the correct title and zero page errors.

The curated model evaluation contains ten generated questions. Keyword and hybrid runs each passed those cases. This is a regression result, not a broad accuracy estimate, an independently held-out study or a measured hybrid improvement. A filename or polished chart cannot create that missing evidence.

Future evaluation should separate retrieval relevance, supported-claim rate, contradictions, appropriate abstention, tool correctness and end-to-end task completion. Use deterministic checks where possible and human review where judgment is required. Validate an LLM judge against human labels rather than treating it as automatic ground truth.

Training comparisons require frozen family/source splits and leakage review. Serving reports require p50/p95 latency, throughput, memory, cold/warm behavior and cost per accepted output. Business reports need actual accepted drafts, manual review effort or appropriately designed search/conversion experiments.

The public documentation was checked for relative links, public language/private-path hygiene, preserved ten-phase structure, SLO arithmetic and license/access consistency. CI configuration and a hosted successful run are distinct evidence. New releases need their own commit-specific checks and observed deployment proof.

8. Deployment, operations, recovery and the future 99% SLO

Local source is the authority. Before editing, compare the active release with the local baseline; reconcile newer remote changes. Build and verify locally, prepare a unique immutable release and checksum manifest, restore the archive locally, then preflight the actual target before promotion. End by proving local/archive/live parity and preserving unrelated services.

A frontend-only release must not package unverified backend work or restart an unchanged API. Runtime rollback preserves database/model volumes and requires schema compatibility. An old image cannot reverse an incompatible migration.

Observed recovery included a populated dump restored into a separate database with exact table payload hashes and restricted-role/RLS probes: 15 test workspaces, 106 source revisions and seven drafts. Actual runtime rollback/forward recovery restored functionality and preserved an approved draft's exact hash. A later frontend-only release did not repeat that full database rehearsal; its rollback path had separate safety regressions.

Future backup targets are RPO at most 24 hours and RTO at most four hours. Current evidence does not establish an automated cadence or either target. A complete policy needs owned off-server destinations, consistent backups, protected configuration/evidence coverage, verified encryption/key recovery, retention/deletion rules, missed-backup alerts and timed independent restoration.

The future availability target is at least 99% good eligible interactive requests over a rolling 30-day window. Eligible traffic and exclusions must be explicit. Capacity-driven rejection of an admitted valid request counts as failure; missing telemetry is unknown coverage. Static delivery and the working API must be measured separately.

The proposed interactive deadline is five seconds, requiring implementation/cancellation and workload validation before measurement. Proposed non-model latency is p95 within one second; durable job acknowledgment is within two seconds. Long AI job outcomes require separate class deadlines after benchmarks.

The request error budget is 1% of eligible requests. At one million requests it permits 10,000 bad events. A full-outage time equivalent is 432 minutes in 30 days, but traffic-weighted and time-weighted budgets are not interchangeable. Four hours would consume about 56% of that time-equivalent allowance.

There is no current measured 99% uptime or contractual SLA. Instrumentation, alert delivery, burn-rate policy and recovery drills must precede that claim. Incident response scopes impact, contains the owned service, preserves sanitized evidence, validates recovery and records the prevention gate. Suspected privacy incidents require a notification assessment without waiting for technical restoration.

Reference: https://sre.google/sre-book/service-level-objectives/

9. Security, privacy, legal applicability and rights

The security boundary includes scoped cookie access, forced tenant RLS, restricted credentials, typed input, bounded work, immutable evidence, stale approval rejection and attempt-fenced jobs. Pinned artifacts, nonroot/read-only containers and secret-free release archives support the supply chain. Scanners and tests are evidence, not a guarantee of no vulnerability.

Report vulnerabilities privately to muhammadzahidulislam2222@gmail.com. Use a safe reproduction and no tokens or another person's records in the first message. No bug-bounty payment, response-time guarantee or independent security certification is offered.

Current workspace data includes product/source text, revisions, drafts, citations, jobs, evaluation results and audit metadata. Search questions are processed for retrieval but not stored as raw query text in application audit records. Hosting logs may contain addresses/request metadata. A hashed rate-limit identifier is not automatically anonymous.

The worker cleans expired live workspace data. Cookie expiry and local-storage clearing do not guarantee immediate erasure from logs/backups. Retention periods and encryption-at-rest coverage are not fully verified. Do not publish a fixed erasure promise without operational evidence.

Before merchant personal data is handled, complete a data map, lawful-purpose/role assessment, processor agreements, subprocessors, transfer review, retention schedule, rights/takedown workflow and tested deletion/restoration controls. Future deletion must invalidate descendants and preserve tombstones for recovery. Removing a training row does not erase learned influence; affected adapters may need retraining or disposal.

GDPR applicability depends on establishment, targeting/monitoring and processing—not simply whether an operator is outside the EU or access is free. US obligations depend on federal/sector and applicable state rules. California thresholds/rights require an actual assessment; children's-data processing requires separate review. Privacy pages alone do not establish compliance.

For relevant personal-data incidents, evaluate notification obligations promptly. GDPR supervisory notification generally has a 72-hour awareness-based requirement where applicable, with a risk-based exception and separate affected-person tests. Obtain competent legal support for the actual facts.

Future generative chat, recommendations or adapted model distribution require EU AI Act role/risk/transparency assessment against the current rules. No current exemption or risk classification is asserted. Preserve model/data provenance and do not invent technical, medical or business-performance claims.

The author’s code and associated documentation are released under MIT, copyright 2026 Zahidul Islam. Preserve the MIT notice and disclaimer when reusing them. Independently licensed software, fonts, models and media retain their respective terms. Full third-party notices cover installed production packages; generated media rights must be handled separately from model licenses.

References:
https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/application-gdpr_en ; https://www.ftc.gov/business-guidance/privacy-security ; https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

10. Future 1M+ users, capacity arithmetic and cost

The one-million-plus active-user objective is a future architecture target, not measured current capacity. Active users, connections, requests/second and CPU jobs must be sized separately. One person can read cached content without an API request; a queued model task can outlive its submitting request.

A stated planning example assumes one million active users making six requests/minute: 100,000 requests/second. If 80% are public content and that portion has a 99% edge hit rate, public origin traffic is 800 requests/second. The 20% private API share remains 20,000 requests/second, for 20,800 total origin requests/second.

If 90% of API requests are reads and 90% of those reads hit a tenant-aware application cache, 1,800 reads and 2,000 writes/second remain. At three database operations per uncached request, that is about 11,400 operations/second before background work. Approval/current-source checks must use authoritative state rather than a stale read cache.

For future model jobs, an assumed 0.001 jobs/user/second yields 1,000 jobs/second. An assumed 0.25 CPU-seconds/job at a 60% utilization ceiling requires at least 417 cores before redundancy. These are planning inputs, not a benchmark; measured service time must replace the assumption. At two-second total job latency, about 2,000 jobs would be in the system.

At 1.2 million connections and an assumed 32 KiB state each, edge state alone is about 36.6 GiB before TLS/OS overhead. At 100,000 requests/second and 10 kB delivered content, sustained traffic is about 2.59 PB over 30 days. Cache hits still consume delivery bandwidth.

Separate public edge delivery, stateless interactive API and asynchronous AI/ingestion workers. Bound connection pools, deadlines, tenant fairness, queue admission and model concurrency. Cache keys include tenant/permissions/source/index/model versions and invalidation.

Scale in measured stages: profile the current envelope, separate bottlenecks, add application replicas, evaluate dedicated data/search tiers, then tenant cells and regional cells. A tenant's transactional approval/evidence state needs a clear write authority. Multi-region requires residency, replication-lag/RPO/conflict policies and actual failover tests.

One small server and free cloud allowances cannot provide this target. Estimate compute, memory, database replicas/IOPS, model hardware, egress, storage, backups, observability, CI and support. Proposed Azure/AWS/GCP roles remain independent bounded deployments outside the synchronous critical path; eligibility and associated costs must be verified before execution.

11. Ten phases with parallel work and deployment

Each phase starts frontend, backend, AI/data, infrastructure and QA/security work together against agreed contracts. Every phase retains deployment, acceptance evidence and rollback. A combined release does not erase a missing cloud/store/human-review gate. Current delivered slices do not mark an entire phase complete.

1. Foundation — UI shell; identity/membership/RBAC; evidence schemas; isolated infra; tenant/role tests

Deployment and evidence: Existing-server foundation and eligible Azure static shell; real allow/deny matrix and eligibility inventory

2. Ingestion — Source/job UX; bounded CSV/PDF/image parsing; versioned corpus; storage/backups; parser/deletion tests

Deployment and evidence: Ingestion/source viewer; attributable records, family/source split and independent restore

3. Shopify/read-only — Connection/sync UX; OAuth/webhook receipts; catalog mapping; AWS validator; replay/order tests

Deployment and evidence: Permitted development-store sync and actual eligible AWS run

4. Retrieval/context — Search/context inspector; filter/permission API; method comparisons; GCP benchmark; injection/isolation tests

Deployment and evidence: Current-version retrieval and actual eligible GCP snapshot benchmark

5. Graph/multimodal — Claim/alt review; temporal relationships/JSON-LD; contradictions/observations; bounded workers; human accessibility review

Deployment and evidence: Evidence impact and deterministic structured-data checks

6. Guarded publishing — Exact diff/approval UX; resumable typed tools; bounded workflow; isolated write worker; stale/replay/unknown-write tests

Deployment and evidence: Approved development-store write, readback and conflict-aware reversal

7. Training — Annotation/results UX; dataset/model registry; LoRA and retrieval training; isolated compute; leakage/regression tests

Deployment and evidence: Actual checkpoint plus base-versus-adapted held-out report

8. Research/ranking — Recommendation/trace UX; simulator; ranking/preferences/distillation; bounded jobs; reward/constraint tests

Deployment and evidence: Executed baseline comparison with gains and regressions

9. Operations — Health/degraded UX; retention/recovery APIs; adversarial/serving benchmarks; backup/canary; failure drills

Deployment and evidence: Measured capacity and timed independent recovery against proposed targets

10. Handover — Final workflows/help; compatibility/export; frozen model/data cards; clean rebuild; independent release review

Deployment and evidence: Reproducible release, operational ownership and actual three-cloud records where eligible

The roadmap requires permitted data and real evidence. Local emulation does not count as a cloud deployment. An unexecuted notebook does not count as training. Automated synthetic labels do not replace independent review. A model may fail to improve and remain unpromoted.

Cloud trials can expire. Export artifacts and retain local reproducibility rather than upgrading automatically or reopening accounts to evade limits. No cloud trial holds the only copy of important data.

Safety gates remain throughout: source permissions, deletion propagation, token lifecycle, tool authorization, parser/egress constraints, approval expiry and hash binding, queue fencing, model supply-chain integrity, compatible migrations and independent recovery. Advanced research such as voice, graph neural networks, GRPO, distributed training and GPU kernels requires its own baseline and resource justification.

The full repository build plan retains the detailed five-lane work, dependencies, deployment, exit criteria and rollback for all ten phases. The current engineering priority is a verifiable public release before resuming the saved phase expansion.

12. Reproducibility, lessons and next decisions

A reviewer should be able to understand the current product without private operational access. The repository documentation covers setup, configuration, current API, architecture, security/threats, privacy/legal applicability, accessibility, data/model licenses, evaluation, deployment/CI, incidents/backups, future scale/cost and the ten-phase plan.

The frontend uses a reproducible npm lock. The supplied backend hash lock targets Linux x86_64 and CPython 3.12; other platforms require separately verified locks. Use a dedicated local database, separate runtime/migration credentials, explicit origins, the real worker and a prewarmed pinned model for required CPU checks.

Hosted CI configuration is not an observed hosted result. Record the run URL and commit after it succeeds. Likewise, release publication is not proof of deployment, and uploaded files are not proof of parity. New releases need candidate-specific gate, review, actual-flow and hash evidence.

Several engineering lessons shaped this design. Resource inventory must include networks and ports, not only free RAM. Read-only containers require every cache/temp path to be tested under the real UID. A lock alone does not establish job ownership; attempt fencing does. Responsive design must be tested with maximum valid input. A citation proves origin, not complete question entailment.

These lessons belong in repeatable checks. A failure should produce a small regression gate and a documented recovery path rather than only a one-time repair. Public reporting should explain the problem, outcome and evidence without exposing access details or confidential incidents.

The next commercial decisions are durable account lifecycle, permitted store integration, reviewed ingestion and truthful retrieval comparison. The next model decisions require a frozen dataset, actual training and held-out results. The next operations decisions require measured telemetry, owned backup schedules and timed restoration. The million-user and 99% targets become claims only after their validation.

Current limitations should remain visible as the project develops: temporary access, no external-store writes, no general entailment guarantee, no broad quality study, no large-scale capacity result and no measured rolling availability. Accurate boundaries make the engineering easier to assess and the next work easier to prioritize.

The project is authored by Zahidul Islam. Product, security, privacy, accessibility and rights questions can be sent to muhammadzahidulislam2222@gmail.com. Contributions follow the repository's rights policy, security reporting and code of conduct. The result should be judged by source, reproducibility and measured behavior, with future ambition stated as future work.